

Relational Database Design : Schema Normalization — Chapter 7

2025 年 11
月

School of Computer Science



北京邮电大学

DBS design

DBMS
independent

Application area/problem
in real world

requirements analysis

Specification of functional requirements

conceptual DBS design

Conceptual DBS schema , i.e. E-R schema

DBMS
dependent

logical DBS design

Initial relational schema generating

**Relational schema
normalizing**

logical DBS schema , i.e. relational data schema

physical DBS design

physical DBS schema about physical storage structure
and access method

Outlines

1. Features of good relational design
2. Functional dependency (FD) theory
3. Normal forms
4. Decomposition using functional dependencies
(FDs)
5. Algorithms for decomposition

困难与挑战

- 概念设计，E-R 图转换，逻辑设计，得到初始关系模式
- 1. 初始关系模式存在关系模式属性间数据依赖 (Dependence) 关系
 - 函数依赖 (functional dependence, FD)
- 2. 直接根据初始关系模式及属性间数据依赖关系构造 DBS，可能会违反 DB 完整性约束，导致 DBS 性能下降
 - 冗余问题 pitfalls
 - 插入问题 pitfalls
 - 更新问题 pitfalls
 - 删除问题 pitfalls

7.1 Good Relational DB Design

- **Logical design** consists of two steps:
 - **generating**: initial relational schema
 - **normalizing**: relational schema
- **Bad design** results in **Pitfalls**:
 - **repetition** of information
 - **inability** to represent certain information

Example: Combined Schemas with Pitfalls

- Suppose to **combine** two tables, **many-to-one** mapping
 - instructor* (*ID*, *name*, *salary*, *dept_name*)
 - department* (*dept_name*, *building*, *budget*)into
 - ins_dept* (*ID*, *name*, *salary*, *dept_name*, *building*, *budget*)
- **Problem:** Possible repetition of information on *department*
 - **many-to-one** mapping cardinality from *instructor* to *department*
 - With ID=45565, 10101, 83831, **values** for attributes *building* and *budget*
- **Pitfalls: Information repetition**
 - inserting
 - deleting
 - updating

	<i>ID</i>	<i>name</i>	<i>salary</i>	<i>dept_name</i>	<i>building</i>	<i>budget</i>
	22222	Einstein	95000	Physics	Watson	70000
	12121	Wu	90000	Finance	Painter	120000
	32343	El Said	60000	History	Painter	50000
→	45565	Katz	75000	Comp. Sci.	Taylor	100000
	98345	Kim	80000	Elec. Eng.	Taylor	85000
	76766	Crick	72000	Biology	Watson	90000
→	10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
	58583	Califieri	62000	History	Painter	50000
→	83821	Brandt	92000	Comp. Sci.	Taylor	100000
	15151	Mozart	40000	Music	Packard	80000
	33456	Gold	87000	Physics	Watson	70000
	76543	Singh	80000	Finance	Painter	120000
t_1	201081,	Zhang,	60000,	Comp. Sci	Taylor	100000
t_2	null,	null,	null,	Soft. Eng.,	Taylor,	100000

Fig. 7.2

Pitfalls

- **Inserting** problem : information redundancy
 - **adding** a new *instructor* 
 - tuple $t_1 = (201081, \text{Zhang}, 60000, \text{Comp.Sci}, \text{Taylor}, 10000)$ is inserted into *inst_dept*
 - Data for *dept_name*, *building* and *budget* are repeated for the *instructors*
- **Deleting** problem
 - if department **Comp.Sci** is canceled, all *instructor* in it should then be removed
 - **every tuples** containing the **Comp.Sci** department should be deleted



Pitfalls

- **Updating** problem : **information redundancy**
 - information redundancy complicates updating, introducing possibility of inconsistency of the *budget* values
 - change *budget* of the **Comp.Sci** department from 100000 to 120000, every tuples belonging to **Comp.Sci** has to be updated

Pitfalls

■ Information Representation Problem

- To represent directly information about a **new-opened** department in which there no exists *instructor*, and a tuple containing *null* values inserted

$t_2 = (null, null, null, \text{Soft. Eng., Taylor, 100000}),$

- *null* values in DB complicate data handling in DBS

Motivation of an improved design:

Decompose *inst_dept* schema
into *inst* and *department*

问题解决：Normalization Principles

- **等价变换**：对**初始关系模式集模式分解**，保证**关系模式**
 - 1. **函数无损连接性** (**lossless join**)
 - 2. **函数依赖保持性** (dependency preservation)
- **Motivation**: **关系模式集**需要规范化 (normalization)

7.2 Decomposition using functional dependencies

- 1. Keys and Functional Dependencies
- 2. 三类特殊函数依赖
 - ▣ trivial, partial, transitive
- 3. Lossless Decomposition

Example 1: Combined Schema Without Repetition

- **Combining** relations, **one to one** mapping,
 - *section* (*sec_id*, *building*, *room_number*)
 - class* (*course_id*, *sec_id*, *semester*, *year*)
 - into **one** relation
 - *sec_class* (*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*)
- **No repetition**, because of **one to one** mapping
 - from *section* to *class* cardinality

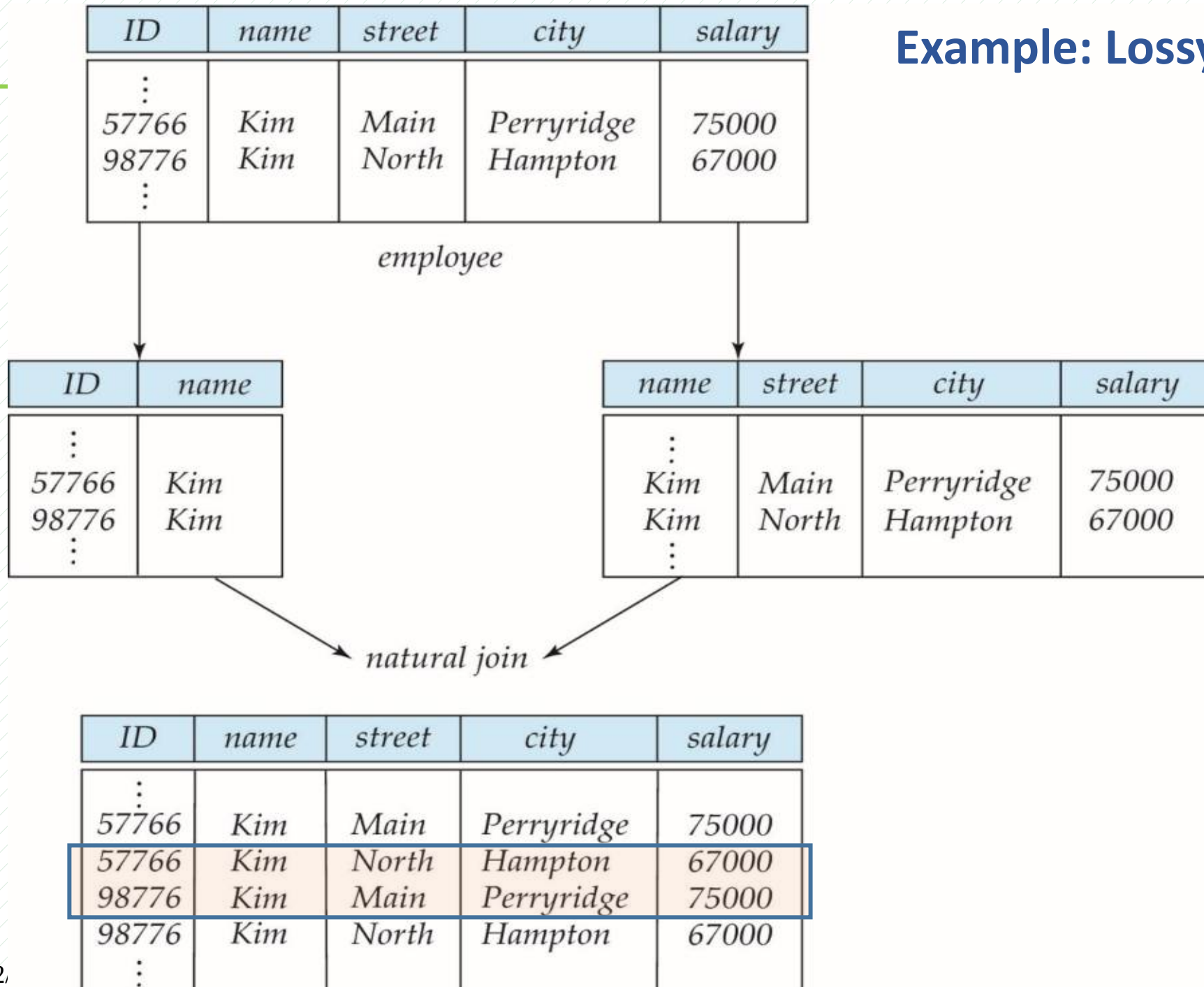
Example 2: Decomposed Smaller Schemas

- Started with *ins_dept* (*ID*, *name*, *salary*, *dept_name*, *building*, *budget*).
- Decompose it into *instructor* and *department*
- Assume if there is a schema (*dept_name*, *building*, *budget*),
then *dept_name* would be a candidate key
- Denote as a functional dependency (FD):
$$\text{dept_name} \rightarrow \text{building, budget}$$
- Note: In *ins_dept*, because *dept_name* is **not** a candidate key,
dept_name, building, budget of **department** have to be **repeated**.

Example 3: Decomposed Smaller Schemas

- **Not** all decompositions are good
- **Example: decompose**
 employee (ID, name, street, city, salary)
 into
 employee 1 (ID, name)
 employee 2 (name, street, city, salary)
- **Problem: information loss**
 - We cannot reconstruct the original **employee** relation

Example: Lossy Decomposition



Example 4: Lossless-Join Decomposition

- Lossless join decomposition

- Decomposition of $R = (A, B, C)$

$$R_1 = (\underline{A}, B) \quad R_2 = (\underline{B}, C)$$

A	B	C
α	1	A
β	2	B

r

A	B
α	1
β	2

$\Pi_{A,B}(r)$

B	C
1	A
2	B

$\Pi_{B,C}(r)$

A	B	C
α	1	A
β	2	B

$$\Pi_A(r) \bowtie \Pi_B(r)$$

Lossless Decomposition

- **Def 1:** The decomposition is a **lossless decomposition**

Let R be a relation schema, R_1 and R_2 form a decomposition of R .

That is $R = R_1 \cup R_2$

If there is **no loss** information by replacing the schema R with $R_1 \cup R_2$

- Formally,

$$\Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) = r$$

- **Def 2:** The decomposition is a **lossy decomposition** if

$$r \subset \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

```
select *  
from (select R1 from r)  
    natural join  
    (select R2 from r)
```

Normalization Principles

- **等价变换 / 模式分解**：对**初始关系模式集**，保证关系模式
 - ✓ 1. 函数无损连接性 (lossless join)
 - 2. 函数依赖保持性 (dependency preservation) ?
- **等价变换 / 模式分解**关系模式集需要规范化处理
- 关系模式规范化主要步骤为
 - 1. 根据**函数依赖** Armstrong 公理系统，
从**初始关系模式集**已知**函数依赖**出发，推导初始关系模式集**所有函数依赖**
 - 2. 对具有**函数依赖**的初始关系模式集，采用**模式分解算法**，
对其进行 (等价) 分解和变换，将其转换为各种范式形式：
 - 1NF 、 2NF 、 BCNF 、 3NF

Normalization Theory

- **Motivation** : Assume an initial relation R is **not** in **good** form, **decompose** R into relations $\{R_1 R_2 \dots R_n\}$ such that
 - **each** relation R_i is in **good** form
 - decomposition is a **lossless-join** decomposition
- **Decomposition** is based on:
 - **functional dependencies (FD)**

CSDN、知乎话题：

关系规范化理论是否有用，数据库设计是否需要？

- 1) 设计要求，存在的问题
- 2) 解决手段，trade-off

Functional Dependencies as Constraints

- **FD as Constraints:**

There are usually (*integrity*) **constraints** (*rules*) on **data**

- ✓ **Example:** constraints that are expected to hold:

- ▣ Students and instructors are **uniquely identified** by their ID.
- ▣ Each student and instructor has **only one name**.
- ▣ Each instructor and student is associated with **only one department**.

Functional Dependencies

- **Def:** a legal instance $r(R)$ of relation R is an instance of a relation that satisfies all such **real-world constraints**
- **Def:** Function: $f: X \rightarrow Y, x \in X, y \in Y, y = f(x)$
 - For $x_1, x_2 \in X$, if $x_1 = x_2$, then $f(x_1) = f(x_2)$
- **Def:** Functional dependency (FD)
 - **Real-world** constraints on the legal relations that the value for a certain set of **attributes** determines uniquely the value for another set of **attributes**
- **Note:** FD is generalization of the **key**

Relation Instance **Satisfy** Functional Dependency

- **Def 1:** For a schema R and $\alpha \subseteq R, \beta \subseteq R$,
a relation instance $r(R)$ satisfy **FD** $\alpha \rightarrow \beta$, or $\alpha \rightarrow \beta$ is satisfied by $r(R)$, if
[关系 $r(R)$ 满足函数依赖 **FD** $\alpha \rightarrow \beta$ 或 $\alpha \rightarrow \beta$ 被 $r(R)$ 满足]
for pairs of tuples t_i and $t_j \in r(R)$ such that
 $t_i[\alpha] = t_j[\alpha] \rightarrow t_i[\beta] = t_j[\beta]$
- **Def 2:** $r(R)$ is **legal** under **FD** $\alpha \rightarrow \beta$, if $\alpha \rightarrow \beta$ is satisfied by $r(R)$
- **Note:** The values for a certain set of attributes **determines uniquely** the value for another set of attributes

SQL Statements for FD Judgment

- 给定关系模式 $R(A, B, C)$ 和关系 $r(R)$,
利用SQL语句判断关系 $r(R)$ 是否满足函数依赖

$$B \rightarrow C$$

- 要求: 如果 $r(R)$ 不满足该函数依赖, 找出导致不满足的 $r(R)$ 中的元组。

- 方法

- 1. 使用元组变量
- 2. group by
- 如果查询结果为null, not exist,
则函数依赖成立

T			S		
A	B	C	A	B	C
1	2	3	1	2	3
1	2	3	1	2	4
1	2	4	1	2	3
1	2	4	1	2	4

```
select *  
from  $r$   
group by  $B$   
having count(distinct  $C$ )>1
```

```
select *  
from  $r$  as  $T$ ,  $r$  as  $S$   
where  $T.B=S.B$   
and  $T.C \neq S.C$ 
```

Example: SQL for FD Maintenance

- 设计一个断言，以保证在 $R(A, B, C)$ 上成立 $B \rightarrow C$

create assertion B $\downarrow$ C check

(not exists

(select B

from R

group by B

having count (distinct C) > 1

)

)

Example

- Given relation $r(R)$ shown below, which FD is satisfied by r

□ A. $A \rightarrow B$ B. $AC \rightarrow B$ C. $BC \rightarrow A$ D. $B \rightarrow C$

	A	B	C
t1	1	4	2
t2	3	5	6
t3	3	4	6
t4	7	3	8
t5	9	1	0

- $t2[A]=t3[A]=3$, $t2[B]=5 \neq t3[B]=4$,
 $A \rightarrow B$ is not satisfied
- $t2[AC]=t3[AC]=36$, $t2[B]=5 \neq t3[B]=4$,
 $AC \rightarrow B$ is not satisfied
- $BC \rightarrow A$ is satisfied
- $t1[B]=t3[B]=4$, $t1[C]=2 \neq t3[C]=6$,
 $B \rightarrow C$ is not satisfied

Functional Dependency Holds on Schema $r(R)$

- **Def:** Let R be a relation schema

$$\alpha \subseteq R \text{ and } \beta \subseteq R$$

The FD **holds on** R ,

函数依赖 **FD** 在关系模式 R 成立

$$\alpha \downarrow \beta$$

if **every** legal instance $r(R)$ satisfies $\alpha \rightarrow \beta$

- **Note:** Whenever two tuples t_1 and t_2 **agree on** the attributes α ,
also **agree on** the attributes β ,

$$t_1[\alpha] = t_2[\alpha] \Rightarrow t_1[\beta] = t_2[\beta]$$

- **Example:** Consider $r(A,B)$ with the following instance of r .

1	4
1	5
3	7

On this instance, $B \rightarrow A$ hold; $A \rightarrow B$ does **NOT** hold,

Example 5: FD Modeling

- Consider

$R = (\text{employee_ID}, \text{date}, \text{turnover per-day}, \text{department_name}, \text{manager}).$

- ✓ Assume that

- every *day*, each *employee* has **only one** *turnover per-day*
- each *employee* works at **only one** *department*
- each *department* is managed by **only one** *manager*

- FDs on R

- $F = \{ \text{employee_ID}, \text{date} \rightarrow \text{turnover per-day},$
 $\text{employee_ID} \rightarrow \text{department_name},$
 $\text{department_name} \rightarrow \text{manager} \}$

FD holds on R vs FD is satisfied by $r(R)$

- There may be **more than one relation instance** $r(R)$, defined on R , i.e.
 $r_1(R)$, $r_2(R)$, $r_3(R)$,..., $r_i(R)$
- **Def:** Relation $r(R)$ satisfies $\alpha \rightarrow \beta$ **vs** $\alpha \rightarrow \beta$ holds on schema R
 - if $\alpha \rightarrow \beta$ holds on R , then **every legal** $r(R)$ satisfies this R
 - **but** for schema R and $\alpha \rightarrow \beta$,
if **only some** $r_i(R)$ satisfies R , $\alpha \rightarrow \beta$ may **not** hold on R .
- For **a relation** $r(R)$ defined on schema R ,
if r satisfies $FD = \{\alpha \rightarrow \beta\}$, then FD holds on schema R
 - **May be false**

Example

- e.g. $A \rightarrow C$ and $AB \rightarrow D$ are satisfied by $r_1(R)$,
but $A \rightarrow C$ is not satisfied by $r_2(R)$,
so $A \rightarrow C$ does not hold on R

✓ $FD_1 = \{A \rightarrow C, AB \rightarrow C\}$, satisfied by r_1

✓ $FD_2 = \{AB \rightarrow C\}$, satisfied by r_2

	A	B	C	D
t1	a_1	b_1	c_1	d_1
t2	a_1	b_2	c_1	d_2
t3	a_2	b_2	c_2	d_2
t4	a_2	b_2	c_2	d_3
	a_3	b_3	c_2	d_4

r_1

	A	B	C	D
t5	a_1	b_1	c_1	d_1
	a_1	b_2	c_1	d_2
	a_2	b_2	c_2	d_2
	a_2	b_2	c_2	d_3
	a_3	b_3	c_2	d_4
t6	a_1	b_4	c_2	d_5

r_2

FD holds on R vs FD is satisfied by $r(R)$

■ FD holds on R

- 定义在 R 的**属性间**的语义约束，或 R 的属性间体现出的语义约束
- 从设计角度，R 应满足的约束

■ FD is satisfied by $r(R)$

- 根据 R 构造的**实际数据** $r(R)$ 是否满足语义约束

1. Keys and Functional Dependencies

- **Def:** K is a **superkey** for relation schema R if and only if $K \rightarrow R$
- **Def:** K is a **candidate key** for R if and only if
 - $K \rightarrow R$, and
 - for **no** $\alpha \subset K$, $\alpha \rightarrow R$
- **Example:** Consider the schema:
in_dep (*ID*, *name*, *salary*, *dept name*, *building*, *budget*).
We expect FD to hold:
 $dept_name \rightarrow building$
 $ID \rightarrow building$
but not to hold:
 $dept_name \rightarrow salary$

FD as Integrity Constraints in DB

- While **E-R** diagrams describe **objects** and **associations** among them, **FD** illustrates **relationships** among **attributes** of objects (i.e. entities or relationships in E-R diagrams)
- How to guarantee FD in DBS?
 - ▣ integrity mechanisms in DBMS, i.e.
 - keys,
 - checks,
 - triggers

Use of Functional Dependencies

- **Note 1: Use of FD**
 - **1. test relations** to see if they are **legal**.
 - If a relation R is legal under an FD F , R **satisfies** F
 - **2. specify constraints** on the set of **legal relations**
 - F **holds on** R if all legal relations on R satisfy the FD set F .
- **Note 2:** A **specific** instance $r_o(R)$ of a relation schema R may satisfy a FD in F even if the FD does **not** hold on **all** legal instances $\{r_i(R)\}$

2. Trivial (平凡) Functional Dependencies

- **Def 1:** A FD is **trivial** if it is satisfied by all instances of a relation:

$\alpha \rightarrow \beta$ is **trivial** if $\beta \subseteq \alpha$

- **Example:**

- $name \rightarrow name$
- $ID, name \rightarrow ID$

- **Def :** Prime attribute (主属性)

- the attribute appears in at least one candidate key

3. Transitive (传递) dependency

- **Def 2:** A functional dependency $\alpha \rightarrow \gamma$ is *transitive*, if // 传递函数依赖
 - $\alpha \rightarrow \beta$ holds on, but $\beta \rightarrow \alpha$ does not hold
 - $\beta \rightarrow \gamma$ holds on, and γ is not in α ,
 - γ is called transitive dependent on α
- e.g. *Student* (sno, sname, address, depart)
 - For the two **FDs**
 $sno \rightarrow sname$,
 $sname \rightarrow address$,
there are *transitive* dependency
 $sno \rightarrow address$

4. Partial (部分) Dependency

- **Def 3:** A FD $\alpha \rightarrow \beta$ is **partial** , β is **partially** dependent on α if there is a **subset** γ of α , i.e. $\gamma \subset \alpha$, such that $\gamma \rightarrow \beta$,

- 部分函数依赖 , α 非最小化 , 有冗余

✓ e.g. *Student (sno, sname, address, depart)*

- For the **partial** dependency

(sno, sname) → address,

there are subsets of attributes such that

sname → address,

sno → address

5. Closure of functional dependency set F^+

- **Def 4:** Use the notation F^+ to denote the **closure** of the FD set F
 - The set of all FDs that can be inferred given the FD set F
 - F^+ contains all of DFs in F
- e.g. $F^+ = \{A \rightarrow B, B \rightarrow C\}^+$
 $= \{A \rightarrow B, B \rightarrow C, A \rightarrow C, AB \rightarrow B, AC \rightarrow BC, \dots\}$

6. Lossless Decomposition

- **Def 1: lossless decomposition**

For $R = (R_1, R_2)$, require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- **Def 2:** A decomposition of R into R_1 and R_2 is **lossless**

if **at least one of** the following DFs is in F^+ :

- $R_1 \cap R_2 \rightarrow R_1$

- $R_1 \cap R_2 \rightarrow R_2$

- **结论 :** If only $R_1 \cap R_2 \rightarrow R_1$ holds, then

$R_1 \cap R_2$ is **the primary key** of R_1 , and **the foreign key** of R_2

- It is a **sufficient condition**

Example

- Consider the schema

in-dep (*ID, name, salary, dept name, building, budget*)

- ✓ Decomposed into the *instructor* and *department* schemas:

instructor (*ID, name, dept_name, salary*)

foreign key!

department (*dept_name, building, budget*)

- Consider the intersection of two schemas, which is *dept_name*

- **Because of**

dept_name → *building, budget*,

the **lossless-decomposition rule** is satisfied.

Example

■ $R = (A, B, C)$ $F = \{A \rightarrow B, B \rightarrow C\}$

✓ $R_1 = (A, B), \quad R_2 = (B, C)$

▣ Lossless decomposition:

$$R_1 \cap R_2 = \{B\} \text{ and } B \rightarrow BC$$

✓ $R_1 = (A, B), \quad R_2 = (A, C)$

▣ Lossless decomposition:

$$R_1 \cap R_2 = \{A\} \text{ and } A \rightarrow AB$$

➤ **Note:**

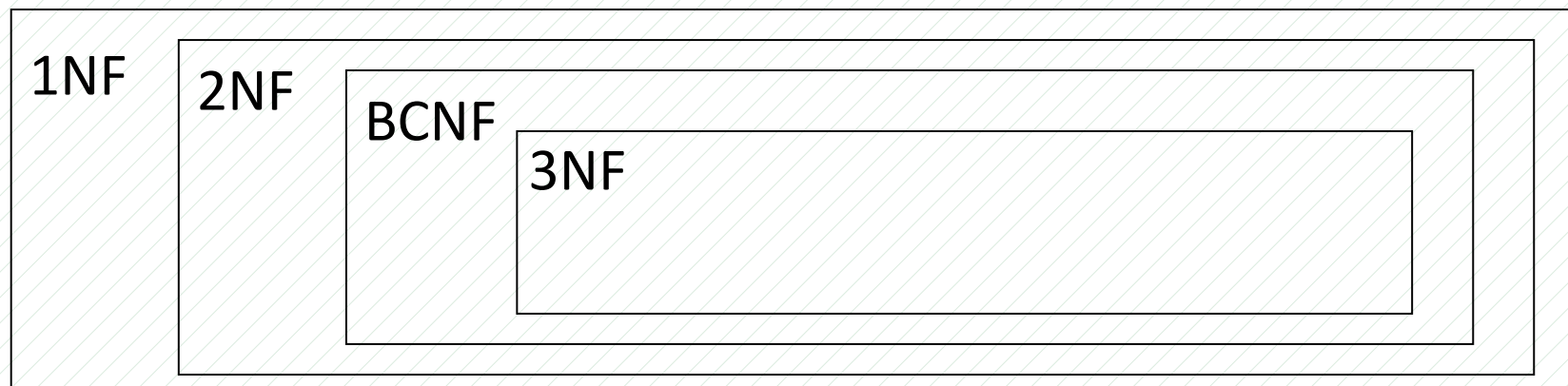
▣ BC is a shorthand notation for $\{B, C\}$

7.3 Normal Forms

- **1NF**: The first Normal Form
- **2NF**: The second Normal Form
- **BCNF**: The **Boyce-Codd** Normal Form
- **3NF**: The third Normal Form

Normalization Principles

- 关系模式规范化处理的**基本要求**为：
 - ▣ **静态关系**具有第一范式形式
 - ▣ **动态关系**最好具有 3NF 或 BCNF 形式
- 范式 1NF、2NF、3NF、BCNF 可以看作由符合范式要求的各种关系模式组成的关系模式的集合
 - ▣ e.g. $1NF = \{ R \mid R \text{ 满足第一范式的定义} \}$
- **范式间的关系**



Atomic Domains and First Normal Form

- **Def** : A relational schema R is in **first normal form** if the domains of all attributes of R are **atomic**
- **Domain is atomic** if its elements are **indivisible** units
 - ▣ examples of **non-atomic** domains:
 - **names**, e.g. student_names={Henry, Codd, ...}
 - **composite attributes**, e.g. [last_name, middle_name, first_name]

Atomic Domains and First Normal Form

- **Def:** Atomicity is a property of how elements of the domain are used
 - Example: Strings would normally be considered **indivisible**
 - Students are given roll numbers which are strings of *CS0012* or *EE1127*
if the first two characters are extracted to find the department,
the domain of roll numbers is not atomic.
- **doing so is a bad idea**
- **Example**

ID	name	age	origin	department	grade
2020 211 3xx	Li	20	Beijing	CS	2019
2019 211 4xx	Wang	21	SD	CS	2018

Second Normal Form

- **Def:** A relation schema R is in **2NF** with respect to a DF set F , if
 - R is in **1NF**, and
 - **Each attribute A** meets one of the criteria
 - it appears in a **candidate key** (a prime attribute)
 A 是主属性
 - it is (not partially) dependent on a **candidate key**
 A 是非主属性，完全依赖于候选键
- **2NF 特点**
 - 不存在非主属性对候选键的部分依赖
 - 非主属性完全依赖于候选键



2NF Examples

- SLC (S# , SDpt, SLocation, C# , Grade)
 - $(S\#, C\#) \rightarrow \text{Grade}$, $SDpt \rightarrow SLocation$,
 $S\# \rightarrow SDpt$, $(S\#, C\#) \rightarrow SDpt$,
 $S\# \rightarrow SLocation$, $(S\#, C\#) \rightarrow SLocation$
 - SLC is not in 2NF, because
 - for non-primary attribute SDpt,
there exists $S\# \rightarrow SDpt$, so SDpt is partially dependent on key $(S\#, C\#)$
 - for non-primary attribute SLocation,
there exists $S\# \rightarrow SLocation$, so SLocation is partially dependent on key $(S\#, C\#)$

2NF Examples

- R (Sno , Sname, Status, City, Pno , Qty)
 - ▣ Sno $\rightarrow$ Sname, Sno $\rightarrow$ Status,
Status $\rightarrow$ City, (Sno, Pno) $\rightarrow$ Qty
- ▣ **R is not in 2NF**, because
 - ▣ for non-primary attributes
Sname and Status are **partially** dependent on the key (Sno, Pno)
- ▣ **Note:** R can be decomposed into **two 2NF** schemas
 - ▣ R₁(Sno , Sname, Status, City)
 - ▣ R₂(Sno, Pno, Qty)

Boyce-Codd Normal Form, BCNF

- **Def 3:** A relation schema R is in **BCNF** with respect to a FD set F if for all FDs in F^+ of the form

$$\alpha \downarrow \beta$$

where $\alpha \subseteq R$ and $\beta \subseteq R$, **at least one** holds:

- ▣ $\alpha \rightarrow \beta$ is **trivial** (i.e., $\beta \subseteq \alpha$)
- ▣ α is a **superkey**

BCNF Example

- **Example:** schema that is **not** in **BCNF**:

in_dep (ID, name, salary, dept_name, building, budget)

Two FDs: $ID \rightarrow name, salary$; $dept_name \rightarrow building, budget$

Because:

- $dept_name \rightarrow building, budget$
 - holds on *in_dep*
 - but *dept_name* is **not a superkey**
- **Note:** **Decompose** *in_dep* **into** *instructor* and *department*
 - *Instructor* (ID, name, salary, *dept_name*) is in BCNF
 - *Department* (dept_name, building, budget) is in BCNF

Decomposing a Schema into BCNF

- Let R be a schema R that is **not** in **BCNF**. Let $\alpha \rightarrow \beta$ that causes **violation** of BCNF.
- Decompose R into:
 - ▣ $(\alpha \cup \beta)$ $\alpha \rightarrow \beta$ 组成**单独模式**
 - ▣ $(R - (\beta - \alpha))$ 从 R 中去掉只出现在 $\alpha \twoheadrightarrow \beta$ 的右部 β 、没有出现在左部 α 的属性
- ▣ $(\alpha \cup \beta)$ and $(R - (\beta - \alpha))$ 存在外键关联
- **Example** : in-dep ($ID, name, salary, dept_name, building, budget$)
 - ▣ $\alpha \twoheadrightarrow \beta$
 - ▣ $\alpha = dept_name, \beta = building, budget$and *in_dep* is replaced by
 - ▣ $(\alpha \cup \beta) = (dept_name, building, budget)$
 - ▣ $(R - (\beta - \alpha)) = (ID, name, dept_name, salary)$

BCNF and DF Preservation

- **Not always** possible to achieve **both** BCNF **and** dependency preservation

- Consider a schema:

dept_advisor (*s_ID*, *i_ID*, *dept_name*)

- With FDs:

i_ID → *dept_name*

s_ID, *dept_name* → *i_ID*

- Two candidate keys =

{*s_ID*, *dept_name*}, {*s_ID*, *i_ID*}

- *dept_advisor* is not in BCNF

- *i_ID* is not a superkey.

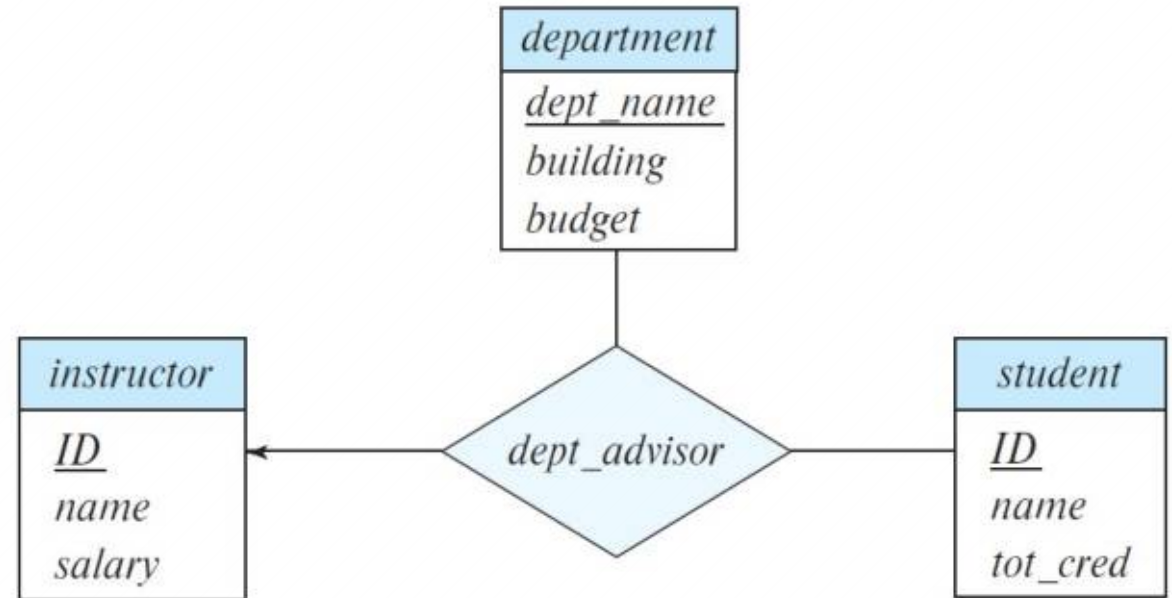


Figure 7.6 The *dept_advisor* relationship set.

Example : Not Dependency Preservation

- We need to decompose *dept_advisor* into two BCNF schemas
 - (*s_ID*, *i_ID*)
 - (*i_ID*, *dept_name*)
- Any decomposition will **not** include all the attributes in
 $s_ID, dept_name \rightarrow i_ID$
- Thus, the composition NOT be dependency preserving
 - the functional dependency
 $s_ID, dept\ name \rightarrow i_ID$
can only be checked by **join of** the decomposed relations

Example

- $R = (A, B, C)$ $F = \{A \rightarrow B, B \rightarrow C\}$
- $R_1 = (A, B), R_2 = (B, C)$
 - Lossless-join decomposition:
 $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
 - Dependency preserving
- $R_1 = (A, B), R_2 = (A, C)$
 - Lossless-join decomposition:
 $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
 - Not dependency preserving
(cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$)

BCNF and DF Preservation

- **Constraints** on **FDs**, are **costly** to check in practice, unless they pertain to **only one** relation
- **If** test only those **DFs** on **each** individual relation of a decomposition to ensure **all** **FDs** hold, then that decomposition is **dependency preserving**
- **Note** : Because it is **not always** possible to achieve both BCNF and dependency preservation, we consider a **weaker normal form**, known as the **third normal form (3NF)**.

Third Normal Form

- A relation schema R is in **third normal form (3NF)** if for all:

$$\alpha \rightarrow \beta \text{ in } F^+$$

at least one of the following holds:

- ▣ $\alpha \rightarrow \beta$ is **trivial** (i.e., $\beta \in \alpha$)
 - ▣ α is a **superkey** for R
 - ▣ **each attribute** in $\beta - \alpha$ is contained in **a** candidate key
出现在右端但不出现在左端的属性
- ▣ e.g. $B \rightarrow ABC$, $(ABC) - (B) = AC$
A appears in a candidate key, and C in another candidate
 - ▣ **NOTE: each attribute may be in a different candidate key**
- Note : If a relation is in BCNF, it is also in 3NF

Third Normal Form

- If R is in 3NF,

2NF 消除非主属性对键的传递函数依赖 → 3NF

- 3NF 性质

- ▣ R is also in 2NF
- ▣ 不存在非主属性对候选键的部分和传递依赖，
- ▣ 每一个非主属性都不传递依赖于 R 的任何候选键

3NF Example

- Consider a schema ***R***:

dept_advisor (*s_ID*, *i_ID*, *dept_name*)

- With FDs:

$i_ID \rightarrow dept_name$

$s_ID, dept_name \rightarrow i_ID$

- Two candidate keys = {*s_ID*, *dept_name*}, {*s_ID*, *i_ID*}

- ***dept_advisor*** is **not** in BCNF

- However, ***dept_advisor*** is in 3NF

- *s_ID*, *dept_name* is a superkey
- $i_ID \rightarrow dept_name$ and *i_ID* is NOT a superkey,
- but:
 - {*dept_name*} – {*i_ID*} = {*dept_name*} and
 - *dept_name* is **contained in a candidate key**

Redundancy in 3NF

- Consider the schema R below, which is in 3NF

- $R = (J, K, L)$
- $F = \{JK \rightarrow L, L \rightarrow K\}$

dept_advisor(s_ID, i_ID, dept_name)
 $s_ID, dept_name \rightarrow i_ID$
 $i_ID \rightarrow dept_name$

- What is wrong with the table?

- ▣ Repetition of information
 - 某个 $L(i_ID)=l_1$ 指导多名学生
- ▣ Need to use null values (e.g., to represent the relationship l_2, k_2)
 - where there is no corresponding value for $J(s_ID)$
 - 某个 $L(i_ID)=l_2$ 没有指导学生

J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

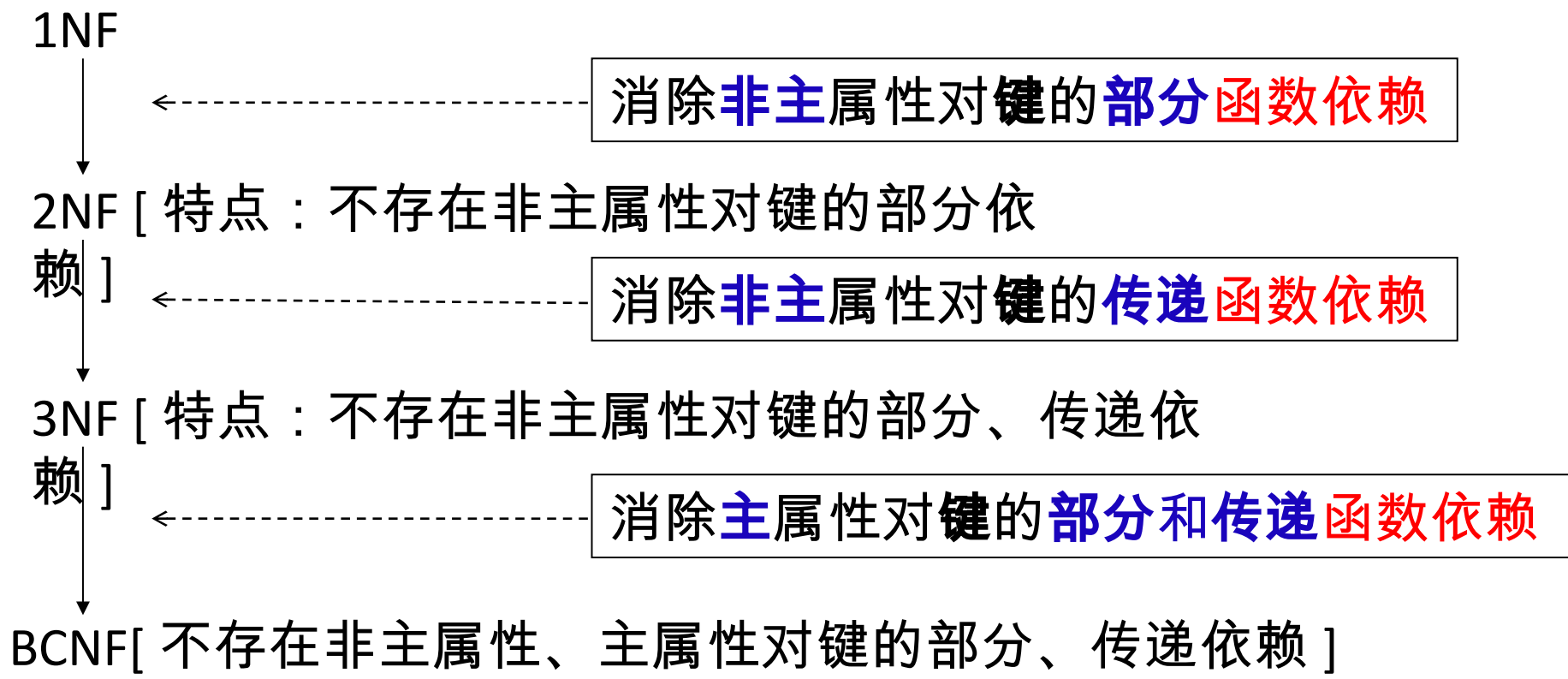
Comparison of BCNF and 3NF

- **Advantages** to 3NF over BCNF. It is always possible to obtain a 3NF design without sacrificing losslessness or dependency preservation.
 - 3NF 分解可以保证无损连接、函数依赖保持
 - BCNF 分解保证无损连接，但不保证函数依赖保持
- **Disadvantages** to 3NF.
 - We may have to use **null** values to represent some of the possible meaningful relationships among data items.
 - There is the problem of **repetition** of information.

Goals of Normalization

- Let R be a relation scheme with a FD set F
Decide whether a relation scheme R is in **good** form
- When a relation scheme R is not in **good** form, decompose it into $\{R_1, R_2, \dots, R_n\}$ such that:
 - ▣ each relation scheme is in **good** form, e.g. in 2NF, 3NF, BCNF
 - ▣ the decomposition is a **lossless decomposition**
 - ▣ the decomposition should be **dependency preserving**

性 函 数 依 赖 保 持



7.4 Functional Dependency (FD) Theory

- ▣ **Closure of FDs**
- ▣ **Closure of Attribute Sets**
- ▣ **Canonical Cover**
- ▣ **Lossless-join Decomposition**
- ▣ **FD Preservation**

Goal

- 1. FDs implied logically by a given set of FDs
 - Armstrong's Axioms
- 2. **Develop** algorithms to **generate** lossless decompositions into BCNF and 3NF
- 3. **Develop** algorithms to **test** if decomposition is FD-preserving

(逻辑蕴含) Logical Implication between FDs

- Given an FD set F , there are **other FDs** f that **logically implied** by F .
 - Example: If $A \rightarrow B$ and $B \rightarrow C$, then infer that $A \rightarrow C$
- **Def.** Given a schema R , a FD f is **logically implied** by a FD set F , if **every** instance $r(R)$ that satisfies F also satisfies f
 - $\{f\} = \{A \rightarrow B, B \rightarrow C\}^+ = \{A \rightarrow B, B \rightarrow C, A \rightarrow C, \dots\}$
- **Def.** The set of **all FDs** logically implied by F is the **closure** of F .
 - Denote the **closure** of F by F^+ .
 - $F^+ = \{f \mid f \text{ is logically implied by } F\}$

Example

$F = \{A \rightarrow B, B \rightarrow C\};$

$f: A \rightarrow C$ is logically implied by F

	A	B	C
t1	1	4	2
t2	3	5	6
t3	4	4	2
t4	7	3	8
t5	9	1	0

Closure of a Set of FDs

- Compute F^+ , the closure of F , by repeatedly applying **Armstrong's Axioms**:
 - **Reflexive rule**: if $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$
 - **Augmentation rule**: if $\alpha \rightarrow \beta$, then $\gamma \alpha \rightarrow \gamma \beta$
 - **Transitivity rule**: if $\alpha \rightarrow \beta$, and $\beta \rightarrow \gamma$, then $\alpha \rightarrow \gamma$
- These rules are
 - **Sound** -- generate only FDs that actually hold
 - **Complete** -- generate all FDs that hold

Closure of FDs (Cont.)

- **Additional rules:**

- **Union rule:** If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds.
- **Decomposition rule:** If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
- **Pseudotransitivity rule:** If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds.

Example of F^+

- $R = (A, B, C, G, H, I)$
 $F = \{ A \rightarrow B \quad A \rightarrow C \quad B \rightarrow H \quad CG \rightarrow H \quad CG \rightarrow I \}$
- Some members of F^+
 - $A \rightarrow H$
 - by transitivity from $A \rightarrow B$ and $B \rightarrow H$
 - $AG \rightarrow I$
 - by augmenting $A \rightarrow C$ with G , to get $AG \rightarrow CG$ and then transitivity with $CG \rightarrow I$
 - $CG \rightarrow HI$
 - by augmenting $CG \rightarrow I$ to infer $CG \rightarrow CGI$,
augmenting of $CG \rightarrow H$ to infer $CGI \rightarrow HI$,
and then transitivity

Example

- Question

Which rule about functional dependencies shown below is right

- ▣ A. if $\alpha \rightarrow \beta$ then $\beta \rightarrow \alpha$
- ▣ B if $A \rightarrow C$, $BC \rightarrow D$ then $AB \rightarrow D$
- ▣ C. if $AB \rightarrow C$ then $B \rightarrow C$
- ▣ D if $\alpha \subseteq \beta$, then $\alpha \rightarrow \beta$

- Answer: B

Closure of Attribute Sets

- **Def:** An attribute B is **functionally determined** by α **under** F
if $\alpha \rightarrow B$ **under** F
 - $\alpha | F \rightarrow B$
- **Def:** Given a set of attributes α , the **closure of α under F** (denoted by α^+) is the set of attributes functionally determined by α **under** F
 - $\alpha^+ = \{\beta \mid \beta \text{ is functionally determined by } \alpha \text{ under } F\}$
 $= \{\beta \mid \alpha | F \rightarrow \beta \}$

Closure of Attribute Sets

- **Algorithm:** compute α^+ , the closure of α , under F

Input: α , F

Output: α^+

$result := \alpha$;

while (changes to $result$) **do**

for each $\beta \rightarrow \gamma$ **in** F **do**

begin

if $\beta \subseteq result$ **then** $result := result \cup \gamma$

end

Example of Attribute Set Closure

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B \ A \rightarrow C$
 $CG \rightarrow H \ CG \rightarrow I$
 $B \rightarrow H\}$
- $(AG)^+$
 - ▣ 1. $result = AG$
 - ▣ 2. $result = ABCG$ ($A \rightarrow C$ and $A \rightarrow B$)
 - ▣ 3. $result = ABCGH$ ($CG \rightarrow H$ and $CG \subseteq AGBC$)
 - ▣ 4. $result = ABCGHI$ ($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- $(AG)^+ = R$, AG is a *superkey* of R
- 问题 : Is AG a candidate key?
 - ▣ 1. Is AG a super key? Yes
 - 1. Does $AG \rightarrow R$? == Is $R \supseteq (AG)^+$
 - ▣ Is any subset of AG a superkey?
 - 1. Does $A \rightarrow R$? == Is $R \supseteq (A)^+$, no
 - 2. Does $G \rightarrow R$? == Is $R \supseteq (G)^+$, no

Uses of Attribute Closure

Uses of the attribute closure algorithm:

■ 1. Testing superkey

- to test if α is a superkey, we compute α^+ and check if α^+ contains all attributes of R , $R = \alpha^+$

■ 2. Testing FDs

- To check if FD $\alpha \rightarrow \beta$ holds (or, in other words, is in F^+), just check if

$$\beta \subseteq \alpha^+$$

- **Note:** compute α^+ by using attribute closure, and then check if it contains β .

Equivalence of FD Set

- **Def.** For two FDs F and G , if

$$F^+ = G^+$$

then F and G are equivalent (等价)

- Example: $F = \{A \rightarrow B, B \rightarrow C\}$ is equivalent to
 $G = \{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

Canonical Cover

- Whenever a user performs an **update** on ***R***, DBS must ensure that the update does not **violate** any FDs.
- All FDs in ***F*** are satisfied in DBS
 - by **executing** check, assertion, trigger, or more complex SQL statements
 - often at very high cost

Canonical Cover

- If an update violates any FDs in the set ***F***, DBS must roll back the update
 - e.g. update Student_grade
set grade = grade +10
grade is required to be between 0 and 100,
how about for updating on the grades as follows: 60, 70, 80, 90, **100**
- checking ***all*** $\alpha \rightarrow \beta$ in ***F*** holding on ***R*** is **time-consuming**
 - ***F*** may have **redundant** dependencies that can be inferred from others
- e.g. $A \rightarrow C$ is **redundant** in: $\{A \rightarrow B, B \rightarrow C, A \rightarrow C\}$

Canonical Cover

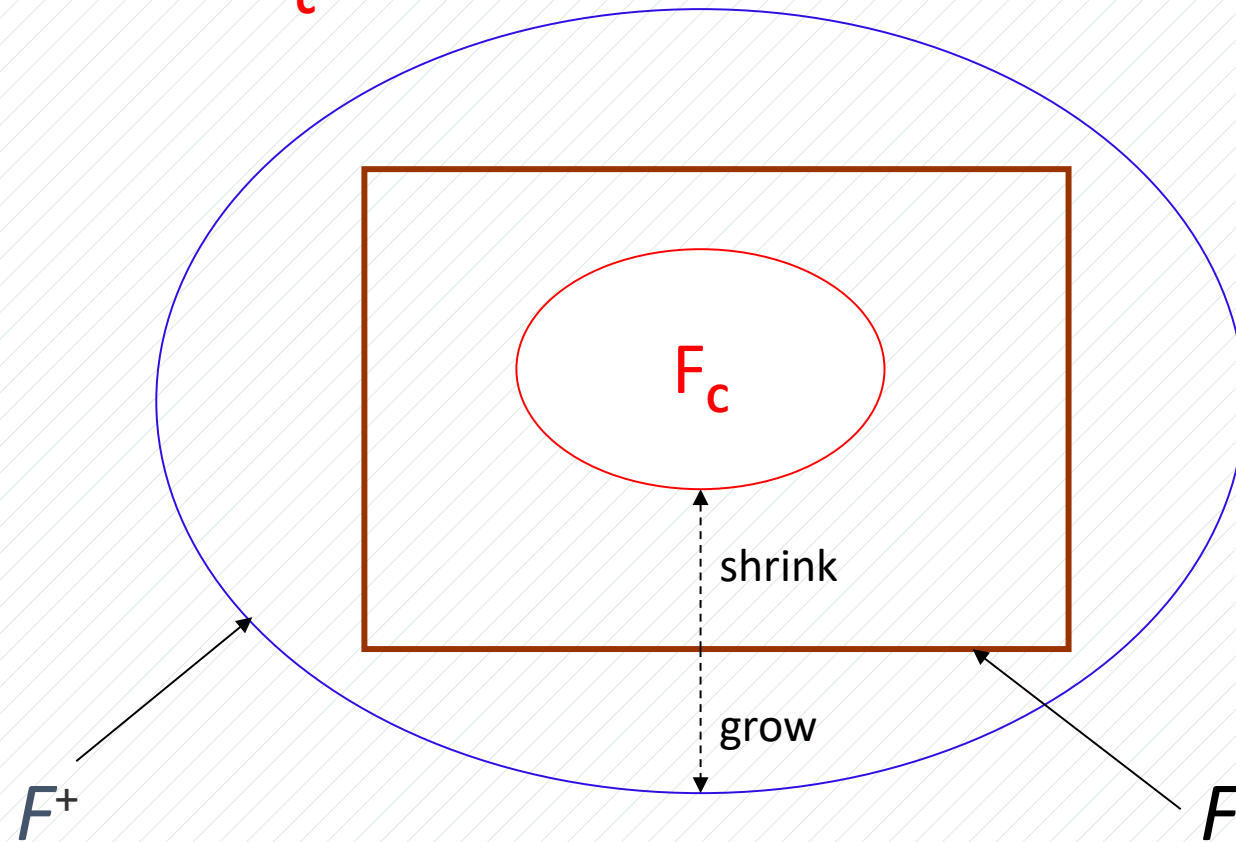
- **Motivation:** The simplified set is termed the **canonical cover**
 - It is desirable to test a **minimal** set of FDs **equivalent** to F
- **Def 1.** a **canonical cover** (正则覆盖) of F , denoted as F_c , is
 - a minimal set of FDs equivalent to F
 - without **redundant** FDs or attributes
 - $F^+ = F_c^+$
- **Def 2: Redundant DFs**

Redundant DFs can be inferred from others in F

 - e.g. $A \rightarrow C$ is redundant: $\{A \rightarrow B, B \rightarrow C, \underline{A} \rightarrow \underline{C}\}$

F , F^+ , and F_c

$$F^+ = F_c^+$$



Extraneous FD and Extraneous Attributes

■ Def 3. extraneous attributes.

- an attribute of FD in F is **extraneous** if we can remove it without changing F^+

■ Motivation: attributes in the left or right side of FD may be extraneous

- LHS: $\{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
 $AC \rightarrow CD : AC \rightarrow D \quad \underline{AC} \rightarrow \underline{C}$ 增加 FD : 强
- RHS: $\{A \rightarrow B, B \rightarrow C, A \rightarrow CD\}$ can be simplified to $\{A \rightarrow B, B \rightarrow C, A \rightarrow D\}$
 $A \rightarrow \cancel{C} \quad A \rightarrow D$ 减少 FD : 弱

Extraneous Attributes

- **性质 1** : Removing an attribute from the **left** side of a FD could make it a **stronger** constraint
 - if we have $AB \rightarrow C$ and remove B , we get the possibly stronger result $A \rightarrow C$
 - because $AB \rightarrow BC$ logically implies $AB \rightarrow C$,
but $AB \rightarrow C$ does not logically imply $A \rightarrow C$!
- **But**, depending on what FD set F happens to be, we may remove B from $AB \rightarrow C$ safely.
 - **Suppose** that
$$F = \{AB \rightarrow C, A \rightarrow D, D \rightarrow C\} \quad \text{弱}$$
Show that F logically implies $A \rightarrow C$, making **extraneous** B in $AB \rightarrow C$.

$$F' = \{A \rightarrow C, A \rightarrow D, D \rightarrow C\}$$

强

Extraneous Attributes (Cont.)

- **性质 2** : Removing an attribute from the **right** side of a FD could make it a **weaker** constraint

- e.g., if we have $AB \rightarrow CD$ and remove **C**, we get the possibly weaker result $AB \rightarrow D$.

- **But**, depending on what DF set **F** happens to be, we may remove C from $AB \rightarrow CD$ safely.

- e.g., suppose that

$$F = \{AB \rightarrow CD, A \rightarrow C\} = \{AB \rightarrow CD, AB \rightarrow BC\}. \text{ 强}$$

After replacing $AB \rightarrow CD$ by $AB \rightarrow D$,

we can still infer $AB \rightarrow C$ and thus $AB \rightarrow CD$

$$F' = \{AB \rightarrow D, A \rightarrow C\} \text{ 弱}$$

Extraneous Attributes

- **性质 3.** Consider a set F of FD and $\alpha \rightarrow \beta$ in F ,
 - 1. attribute A is **extraneous in α** , if
 - $A \in \alpha$, and
 - F implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\} = F'$

i.e., $F^+ \supseteq (F')^+$ **弱 > 强**

- 2. attribute B is **extraneous in β** , if
 - $B \in \beta$, and
 - the set of functional dependencies

$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - B)\}$ implies F

i.e., $(F')^+ \supseteq F^+$ **弱 > 强**

Note:

implication in opposite direction is trivial ,
since a stronger FD always implies
a weaker one

Testing if an Attribute is Extraneous

- Let R be a relation schema and let F be a set of FDs that hold on R

Consider an attribute B in the FD

$$\alpha \twoheadrightarrow \beta$$

- 方案 1:** Test if attribute $B \in \beta$ is extraneous in β

$$\alpha \rightarrow B \text{ under } F' ?$$

- Consider the set:

$$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - B)\},$$

- check that α^+ contains B , $B \in \alpha^+$ under F' ?
- if it does, B is extraneous in β

Testing if an Attribute is Extraneous

- **方案 2:** Test if attribute $A \in \alpha$ is extraneous in α
 $(\alpha - A) \rightarrow \beta$ under F ?
 - Let $\gamma = \alpha - \{A\}$. Check if $\gamma \rightarrow \beta$ can be inferred from F
 - Compute γ^+ using the DFs in F
 - If $\gamma^+ = (\alpha - A)^+$ includes all attributes in β , i.e. $\beta \in (\alpha - A)^+$, then, A is extraneous in α

Examples of Extraneous Attributes

- **Example 1:** Given $F = \{A \rightarrow C, AB \rightarrow C\}$
 - B is extraneous in $AB \rightarrow C$, because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$
- **Example 2:** Given $F = \{A \rightarrow C, AB \rightarrow CD\}$
 - C is extraneous in $AB \rightarrow CD$ since $AB \rightarrow C$ can be inferred even after deleting C
- **Example 3:** Let $F = \{AB \rightarrow CD, A \rightarrow E, E \rightarrow C\}$

To check if C is extraneous in $AB \rightarrow CD$

 - 步骤 1: Compute the closure of AB under $F' = \{AB \rightarrow D, A \rightarrow E, E \rightarrow C\}$
 - 步骤 2: The closure is $ABCDE$, which includes C
 - 结论 : C is extraneous

Canonical Cover

■ **Def 4.** a **canonical cover** (正则覆盖) of F , denoted as F_c , is

- a minimal set F_c of FDs equivalent to F
 - **without** redundant **FDs** or **attributes**
 - $F^+ = F_c^+$

■ 性质 : A **canonical cover** for F is a set of FDs F_c such that

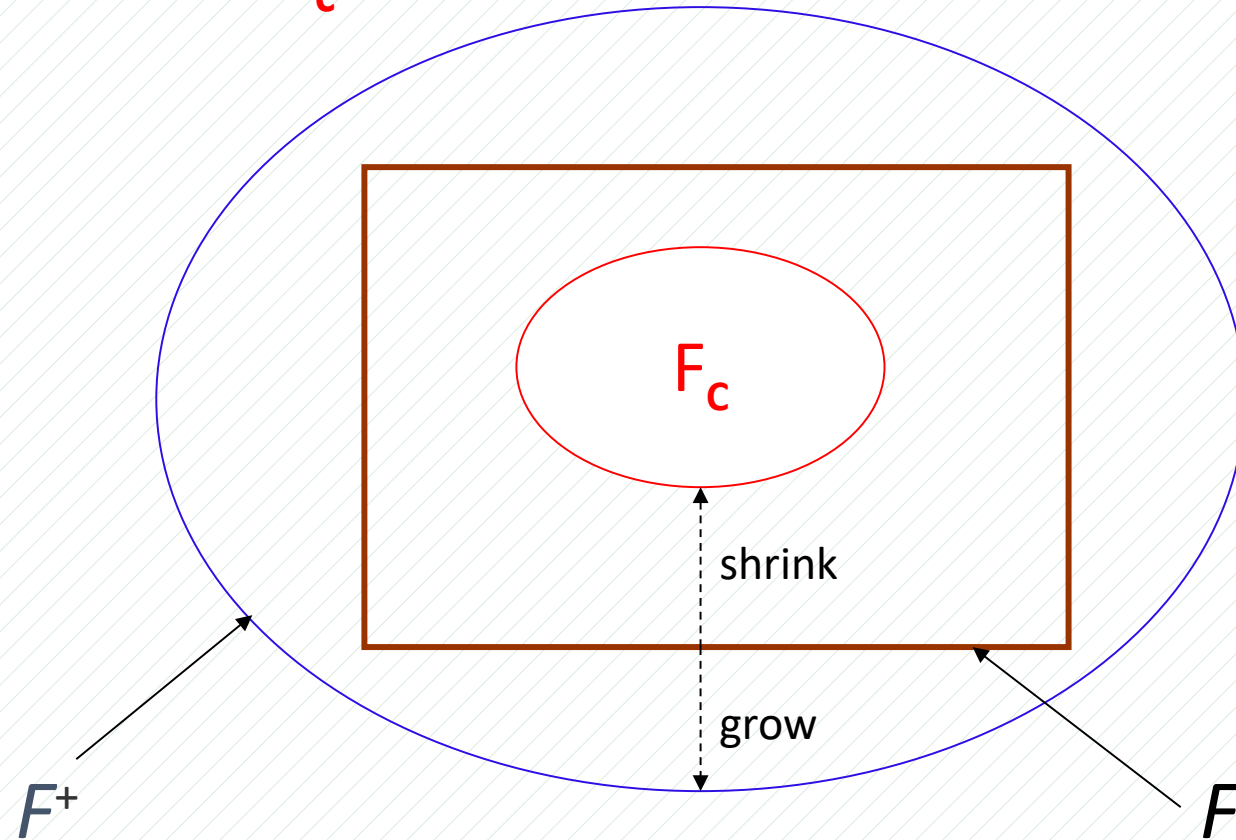
- 1. F logically implies all dependencies in F_c , and
- 2. F_c logically implies all dependencies in F , and
- 3. F_c contains **no extraneous** attributes
- 4. **Left** side of FD in F_c is **unique**.
 - There are no two FDs in F_c

$\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ such that $\alpha_1 = \alpha_2$

$$F^+ = F_c^+$$

F , F^+ , and F_c

$$F^+ = F_c^+$$



Canonical Cover

- **Compute:** a canonical cover for F :

repeat

Use the **union rule** to replace any DFs in F of the form

$$\alpha_1 \rightarrow \beta_1 \text{ and } \alpha_1 \rightarrow \beta_2 \text{ with } \alpha_1 \rightarrow \beta_1 \beta_2$$

Find a FD $\alpha \rightarrow \beta$ in F_c with an **extraneous attribute** either in α or in β

If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$

until F_c not change

Canonical Cover

- **Note 1:** Union rule may become applicable,
after some extraneous attributes have been deleted
- **Note 2:** There may be **several** F_c for a set F of FDs,

Example 4: Computing a Canonical Cover

- $R = (A, B, C)$
 $F = \{ A \rightarrow BC, B \rightarrow C, A \rightarrow B, AB \rightarrow C \}$
- 步骤 1: Use **union rules** to combine $A \rightarrow BC$ and $A \rightarrow B$ into $A \rightarrow BC$
 - Set becomes $F_1 \{A \rightarrow BC, B \rightarrow C, AB \rightarrow C\}$
- 步骤 2: **A is extraneous** in $AB \rightarrow C$
 - Check if the result of **deleting A from** $AB \rightarrow C$ is implied by other dependencies
 - Yes: $B \rightarrow C$ is already present!
 - Set becomes $F_2 \{A \rightarrow BC, B \rightarrow C\}$
- 步骤 3: **C is extraneous** in $A \rightarrow BC$
 - Check if $A \rightarrow C$ is logically implied by $A \rightarrow B$ and the other dependencies
 - Yes: using transitivity on $A \rightarrow B$ and $B \rightarrow C$.
- 结论 : The canonical cover is: $F_c \{A \rightarrow B, B \rightarrow C\}$

Lossless-join Decomposition

- For $R = (R_1, R_2)$, we require that for all possible relations r on schema R

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r)$$

- Decomposition of R into R_1 and R_2 is lossless join if at least one of the following dependencies is in F^+ :

不同子表间存在外键关联

- ▣ $R_1 \cap R_2 \rightarrow R_1$
- ▣ $R_1 \cap R_2 \rightarrow R_2$
- ▣ Note: The above FDs are a **sufficient condition** for lossless join decomposition;
The dependencies are a **necessary condition** only if all constraints are FDs

Motivation

- $R = (A, B, C)$
 $F = \{A \rightarrow B, B \rightarrow C\}$
 - ▣ Can be decomposed in two different ways
- $R_1 = (A, B), R_2 = (B, C)$
 - ▣ Lossless-join decomposition:
 $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
 - ▣ **Dependency preserving**
- $R_1 = (A, B), R_2 = (A, C)$
 - ▣ Lossless-join decomposition:
 $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
 - ▣ **Not dependency preserving**
(cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$)

Dependency Preservation

- **Def 1:** For a schema R , F holds on R , and decomposition $\{R_1, R_2, \dots, R_n\}$ of R , **restriction of F to R_i** , denoted as F_i , is defined as

$$F_i = \{\alpha \rightarrow \beta \mid \alpha \rightarrow \beta \in F^+ \text{ AND } \alpha\beta \subseteq R_i\}$$

- The set of FDs in F^+ that include only attributes in R_i 限制 (投影)

- **Def 2:** Let F_i be **restriction of F to R_i** , i.e. FDs that include attributes in R_i

- A decomposition is **dependency preserving**, if

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+$$

- **Note :** Testing for dependency preservation take exponential time.

Example 1: Dependency Preserving

- $R(A, B, C, D)$, $F = \{A \rightarrow B, B \rightarrow C, A \rightarrow D, B \rightarrow D\}$ on R

- $F^+ = F \cup \{A \rightarrow C\} \cup \dots$

- lossless decomposition:

$$R_1(A, B, C), \quad F_1 = \{A \rightarrow B, B \rightarrow C\} \text{ on } R_1,$$

$$R_2(B, D), \quad F_2 = \{B \rightarrow D\} \text{ on } R_2$$

- Note: $A \rightarrow D$ is lost on R_1 and R_2

- **Problem: Whether $A \rightarrow D$ is preserved?**

- $A \mid F_1 \rightarrow B, B \mid F_2 \rightarrow D,$

so $A \rightarrow D$ is preserved

Testing: Dependency Preservation

- Whether **FD** $\alpha \rightarrow \beta$ is **preserved** in decomposition $R = \{R_1, R_2, \dots, R_n\}$

- **result** = α

repeat

for each R_i in the decomposition

$$t = (\text{result} \cap R_i)^+ \cap R_i$$

$$\text{result} = \text{result} \cup t$$

until (**result** does not change)

- If **result** contains all attributes in β , FD $\alpha \rightarrow \beta$ preserved.

- Test on **all FDs** to check **whether** decomposition is dependency preserving

- decomposition is preserved, if and only if

all FDs $\alpha \rightarrow \beta$ are preserved.

- This procedure takes **polynomial time**

从 α 出发，利用只包含在各子模式 R_i
中间结果 **result** 去推导 β
推导：函数确定 ↓

Example 1

- $R = (A, B, C)$
 $F = \{A \rightarrow B, B \rightarrow C\}$
Key = $\{A\}$
- R is not in BCNF
- Decomposition $R_1 = (A, B), R_2 = (B, C)$
 - ▣ R_1 and R_2 in BCNF
 - ▣ Lossless-join decomposition
 - ▣ Dependency preserving

Example 2

- *Student* (*sno*, *dept*, *head*),
 $F = \{sno \rightarrow dept, dept \rightarrow head\}$,
 $F^+ = F \cup \{sno \rightarrow head\} \cup \{\dots\}$
- **Decomposition 1:**
 $R_1(sno, dept), F_1 = \{sno \rightarrow dept\}$
 $R_2(sno, head), F_2 = \{sno \rightarrow head\}$
 - **lossless**, because
 - $R_1 \cap R_2 = \{sno\}$, the key of R_1 and R_2
 - **non-dependency preservation**, because
 - $(F_1 \cup F_2)^+ \neq F^+$, $dept \rightarrow head$ is **lost**,

- 原因 : for $dept \rightarrow head$ in F ,
 - (1) with respect to R_1 ,
 $result = (dept \cap R_1)^+ \cap R_1$
 $= \{dept\}^+ \cap \{sno, dept\}$
 $= \{dept, head\} \cap \{sno, dept\}$
 $= \{dept\};$

- (2) with respect to R_2 ,
 $result = (dept \cap R_2)^+ \cap R_2$
 $= \phi^+ \cap \{sno, head\} = \phi$

- 结论 : $dept \rightarrow head$ is not preserved

Example 2

□ **Decomposition 2:**

$R_1(sno, dept), \quad F_1 = \{sno \rightarrow dept\}$

$R_2(dept, head), \quad F_2 = \{dept \rightarrow head\}$

□ **lossless-join**, because

– $R_1 \cap R_2 = \{dept\}$, and is the key of R_2

□ **dependency preservation**, because

– $(F_1 \cup F_2)^+ = F^+$

7.5 Algorithms for Decomposition Using FD

- **Testing BCNF**
- **Decomposition for BCNF**
- **Algorithm for BCNF decomposition**

- **Testing 3NF**
- **Decomposition for 3NF**
- **Algorithm for 3NF decomposition**

Testing 1: BCNF

- Whether FD $\alpha \rightarrow \beta$ causes a violation of BCNF
 - 1. compute α^+ (the attribute closure of α),
 - 2. verify that it includes all attributes of R , that is, it is a superkey of R .

α 判断是否是 superkey

- **Simplified test:**

To check if a relation schema R is in BCNF,
it suffices to check **only one** FD in F for **violation**,
rather than checking all FDs in F^+ .

- if **none of the FDs in F** causes a violation of BCNF,
none of the FDs in F^+ will cause a violation of BCNF either.

Testing Decomposition: BCNF

- **Simplified test for decomposition:**
- **However:** when testing a relation in **decomposition** of R
simplified test using only F is incorrect
 - Consider $R = (A, B, C, D, E)$, with $F = \{A \rightarrow B, BC \rightarrow D\}$
 - Decompose R into $R_1 = (A, B)$ and $R_2 = (A, C, D, E)$
 - Neither of FDs in F contain only attributes from (A, C, D, E)
so we might be misled into thinking R_2 satisfies BCNF
 - In fact, FD $AC \rightarrow D$ in F^+ shows R_2 is not in BCNF

Testing Decomposition: BCNF

- Testing: **Whether relation R_i in decomposition of R is in BCNF ?**
- **Decomposition Algorithm** for **lossless** schema in BCNF
 - ▣ determine whether α is the superkey of subschema R_i
- ▣ **Either** test R_i with respect to F_i , **restriction** of F^+ to R_i
- ▣ **Or** use the **original** set of FDs F that hold on R , **but** with test:
 - for every set of attributes $\alpha \subseteq R_i$, with respect to F_i , **check** that α^+ **either** includes no attribute of $R_i - \alpha$, **or** includes all attributes of R_i .

Given: R , F holding on R

result := { R };

done := false;

while (not **done**) **do**

if (there is a **non**-BCNF subschema R_i in **result**)

then begin

let $\alpha \rightarrow \beta$ be an FD that holds on R_i , such that

(1) α is not the **superkey** for R_i , due to $\alpha \rightarrow \beta$

with respect to the restriction F_i to R_i !!!

(2) $\alpha \cap \beta = \emptyset$; $\alpha \rightarrow \beta$ 中无冗余属性

result := {(**result** - R_i)} $\cup$ {(R_i - β)} $\cup$ {(α , β)};

end

else done := true

1. 找出 non-BCNF 子模式 R_i ,
进一步分解

R_i is non-BCNF
due to $\alpha \rightarrow \beta$

2. 用分解后的子模式 ($\alpha\beta$) 和 ($R_i - \beta$) 替
换 **result** 中不满足 BCNF 的 R_i

α remains in R_i , R_i 中除去右端 β 的剩余部分

BCNF decomposition algorithm

■ Note :

- whether R_i is in BCNF ?
- R_i is not in BCNF,
because of $\alpha \rightarrow \beta$ on R_i , α is not the super key of R_i
- the algorithm replaces non-BCNF R_i with $(R_i - \beta)$ and (α, β)
 - R_i is decomposed into $(R_i - \beta)$ and BCNF (α, β)
- the restriction of F to the schema (α, β) is $\alpha \rightarrow \beta$,
and α is the superkey for (α, β) ,
so (α, β) is in BCNF

Example 1

- $R = (A, B, C)$
 $F = \{A \rightarrow B, B \rightarrow C\}$
Key = $\{A\}$
- R is not in BCNF ($B \rightarrow C$ but B is not superkey)
- Decomposition
 - ▣ $R_1 = (\underline{B}, C)$
 - $F_1 = \{B \rightarrow C\}$
 - ▣ $R_2 = (\underline{A}, B)$
 - $F_2 = \{A \rightarrow B\}$

Example 2

- *class* (*course_id*, *title*, *dept_name*, *credits*,
sec_id, *semester*, *year*,
building, *room_number*, *capacity*,
time_slot_id)

- FDs:

- *course_id* → *title*, *dept_name*, *credits*
- *building*, *room_number* → *capacity*
- *course_id*, *sec_id*, *semester*, *year*
→ *building*, *room_number*, *time_slot_id*

- Candidate key

{*course_id*, *sec_id*, *semester*, *year*}.

course + classroom + section



使用补充算法计算候选键：

$L = \{course_id, sec_id, semester, year\}$

$R = \{title, dept_name, credits, capacity, time_slot_id\}$

$LR = \{building, room_number\}$

$L^+ = R$

Example 2

■ BCNF Decomposition:

- $course_id \rightarrow title, dept_name, credits$ holds
 - but $course_id$ is not a superkey.
- replace **class** by:
 - **course** ($course_id, title, dept_name, credits$)
 - **class-1** ($course_id, sec_id, semester, year, building, room_number, capacity, time_slot_id$)

□ **course** is in BCNF

- How do we know this?
- with $F_{course} = \{course_id \rightarrow title, dept_name, credits\}$ holds on *course*,
and $course_id$ is the candidate key !

Example 2

- **class-1** (*course_id, sec_id, semester, year, building, room_number, capacity, time_slot_id*)
 - $F_{\text{class-1}} = \{ \text{building, room_number} \rightarrow \text{capacity, course_id, sec_id, semester, year} \rightarrow \text{building, room_number, time_slot_id} \}$
 - **candidate key**: {*course_id, sec_id, semester, year*}
- **class-1** is not in BCNF
 - *building, room_number* $\rightarrow$ *capacity* holds on *class-1*, but {*building, room_number*} is not a superkey
 - replace **class-1** by:
 - **classroom** (*building, room_number, capacity*)
 - **section** (*course_id, sec_id, semester, year, building, room_number, time_slot_id*)
- **classroom** and **section** are in BCNF
- **class** is decomposed into **course**, **classroom**, and **section**

使用补充算法计算候选键：

$L = \{ \text{course_id, sec_id, semester, year} \}$

$R = \{ \text{capacity, time_slot_id} \}$

$LR = \{ \text{building, room_number} \}$

$L^+ = R$

$F_{\text{classroom}} = \{ \text{building, room_number} \rightarrow \text{capacity} \}$

$F_{\text{section}} = \{ \text{course_id, sec_id, semester, year} \rightarrow \text{building, room_number, time_slot_id} \}$

Motivation: Example 3

- It is **not always possible** to get a BCNF decomposition **with dependency preserving**
- $R = (J, K, L)$
 $F = \{JK \rightarrow L, L \rightarrow K\}$
Two candidate keys: JK and JL
- R is not in BCNF, due to $L \rightarrow K$
- A BCNF decomposition
 $R_1 = \{L, K\}$ with $L \rightarrow K$,
 $R_2 = \{J, L\}$ with **no** FD, **$JK \rightarrow L$** being lost
- **Problem: No dependency preserving**
Any decomposition of R will fail to preserve **$JK \rightarrow L$**

(s_ID, i_ID, dept_name)		
J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

Decomposition 2: the Third Normal Form (3NF)

Motivation: Third Normal Form

- There are some situations where
 - ▣ BCNF is not dependency preserving,
 - ▣ Checking for FD violation is important
- **Solution:** define a weaker normal form, 3NF
 - ▣ allows some redundancy
 - ▣ There is a **lossless-join, dependency-preserving** decomposition into 3NF
- Note: The candidate keys of R should be computed !!!

Example 1: 3NF

■ Relation *dept_advisor*:

□ *dept_advisor* (*s_ID*, *i_ID*, *dept_name*)

$F = \{s_ID, dept_name \rightarrow i_ID,$
 $i_ID \rightarrow dept_name\}$

□ Candidate keys: $\{s_ID, dept_name\}, \{i_ID, s_ID\}$

□ *R* is not in BCNF, due to: $i_ID \rightarrow dept_name$

□ *R* is in 3NF, because

– $s_ID, dept_name \rightarrow i_ID$

$\{s_ID, dept_name\}$ is a **superkey**

– $i_ID \rightarrow dept_name$

$dept_name$ is contained in $\{s_ID, dept_name\}$,

(*s_ID*, *i_ID*, *dept_name*)

<i>J</i>	<i>L</i>	<i>K</i>
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

1 个 *i_id* 指导多个 *s_ID*, 多个 *s_ID* 属于同一个 *dept_name*

Redundancy in 3NF

- **缺点** : Redundancy

- **Problems** due to redundancy in 3NF

- $R = (J, K, L)$

- $F = \{JK \rightarrow L, L \rightarrow K\}$

- $\underline{J}: s_ID, \underline{L}: i_ID, \underline{K}: dept_name$

- **Repetition** of information (e.g., the relationship l_1, k_1)

- $(i_ID, dept_name)$

- **Null values** (e.g., to represent the relationship

- l_2, k_2 : there is no corresponding value for J .

(s_ID, i_ID, dept_name)		
J	L	K
j_1	l_1	k_1
j_2	l_1	k_1
j_3	l_1	k_1
null	l_2	k_2

1 个 i_id 指导多个 s_ID, 多个 s_ID 属于同一个 dept_name

Testing: 3NF

- **Optimization**

- check **only** FDs in F , **not all** FDs in F^+
- 1. If α is a superkey, use **attribute closure** to check each FD $\alpha \rightarrow \beta$.
- 2. If α is not a superkey, check if **each attribute in β** is contained in **a candidate key**
 - Testing is **expensive**, since it involve finding **all** candidate keys
 - **Testing** for 3NF has been shown to be NP-hard
 - **Decomposition** into 3 NF can be done in polynomial time

Decomposition Algorithm 2: 3NF

- **Decomposition Algorithm:**
lossless and dependency preserving schemas in 3NF
- **Note:**
 - 1. **all** candidate keys should be founded out
 - 2. **only one** F_c should be computed

0. Find out **all** candidate keys for R

1. Find out a canonical cover F_c for F ;

求最简 FD:
去除掉传递依赖
和冗余属性

2. Do for each FD $\alpha \rightarrow \beta$ in F_c

if none of the schemas R_j , $1 \leq j \leq i$ contains $\alpha \beta$

then begin

$i := i + 1$;

$R_i := \alpha \beta$

end

对 F_c 中每一个未包括在已有子模式中的
函数依赖，构造一个 3NF 子模式

说明：只要有一个候选键包含在第 2 步构造出的某个子模式 R_i 中，

就无需在第 3 步中，为候选键单独构造子模式

3. if none of the schemas R_j , $1 \leq j \leq i$ contains a candidate key for R

then begin

$i := i + 1$;

$R_i :=$ any candidate key for R;

任意挑选 R 的一个候选键【只需挑选一个】，对
该候选键单独构造一个关系模式

end

4. return $(R_1, R_2, \dots, R_i)$

Example 1

- Consider schema $R(X, Y, Z, W)$, and $F = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\}$ holds on R .
Give the lossless, dependency-preserving decomposition into 3NF

- The solution:**

- Step 1.** $\{YW\}^+ = R$, $\{YW\}$ is the unique candidate key
- Step 2.** With respect to $F = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\}$,

- considering X in $Y \rightarrow XZ$ and Z in $W \rightarrow XZ$,

$\{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow X\}$ implies F ,

so X in $Y \rightarrow XZ$ and Z in $W \rightarrow XZ$ are all extraneous attributes,

and one of **canonical covers** for F is

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow X\}$$

使用补充算法计算候选键：

$L = \{Y, W\}$

$LR = \{X, Z\}$

$$\begin{aligned} F &= \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\} \\ &= \{X \rightarrow Z, Z \rightarrow X, W \rightarrow X, Y \rightarrow Z, \\ &\quad \underline{Y \rightarrow X}, \underline{W \rightarrow Z}\} \end{aligned}$$

Example 1

- other **canonical covers** for F are

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow Z\},$$

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow X\},$$

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow Z\}$$

- **Step 3.** As far as $F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow X\}$ is concerned,

- on the basis of 3NF decomposition algorithm,
the subschema responding to each functional dependency in F_c are

$$R_1 = \{XZ\}, R_2 = \{YZ\}, R_3 = \{XW\}$$

- for the candidate key $\{YW\}$, $R_4 = \{YW\}$ is generated,
so **one of** the 3NF decomposition is

$$\{XZ, YZ, XW, YW\}$$

Example 1

- Note: With respect to other **canonical covers**

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow X\}, \text{ or}$$

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow Z\}, \text{ or}$$

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow Z\}$$

and the candidate key $\{YW\}$,

- The corresponding 3NF decomposition are

$$\{XZ, YX, WX, YW\}$$

$$\{XZ, YX, WZ, YW\}$$

$$\{XZ, YZ, WZ, YW\}$$

Example 2

- Consider **R** on *student* and FD **F** :

- **R** (stNo, Name, Telno, stId, Classno, Cno, Cname, Grade)

- **F** = { stNo → Name, Telno, stId, Classno,
stId → stNo,
stNo, Cno → Grade,
Cno → Cname }

- **All candidate keys**

- (stNo, Cno), (stId, Cno)

- **Note:** What is the highest normal form of R?

- not in BCNF, because of FD1, FD2, and FD4

- not in 3NF, because FD1,

each attribute in $\beta - \alpha = \{\text{Name, Telno, stId, Classno}\}$ is **not** contained in candidates

补充算法计算候选键 : **Appendix 7-1**

$L = \{\text{Cno}\}$

$LR = \{\text{stNo, stId}\}$

$R = \{\text{Name, Telno, Classno, Cname, Grade}\}$

$(\text{Cno, stNo})^+ = R$

$(\text{Cno, stId})^+ = R$

Example 2

■ Decomposition lossless-join?

$\rho = \{R_1(\text{stNo}, \text{Name}, \text{Telno}, \text{stId}, \text{Classno}), R_2(\text{stId}, \text{Cno}, \text{Cname}, \text{Grade})\}$

- is a lossless-join decomposition, because $R_1 \cap R_2 = \text{stId} \rightarrow R_1$

■ Give a lossless-join and dependency-preserving decomposition of R into 3NF

- $F_c = F$

- 3NF decomposition

- $R_1(\text{stNo}, \text{Name}, \text{Telno}, \text{stId}, \text{Classno})$

- $R_2(\text{stNo}, \text{Cno}, \text{Grade})$

- $R_3(\text{Cno}, \text{Cname})$

- ~~$R_4(\text{stId}, \text{Cno})$~~

F=

{
 $\text{stNo} \rightarrow \text{Name}, \text{Telno}, \text{stId}, \text{Classno},$
 $\text{stId} \rightarrow \text{stNo},$
 $\text{stNo}, \text{Cno} \rightarrow \text{Grade},$
 $\text{Cno} \rightarrow \text{Cname}$
}

Candidate keys:

(stNo, Cno)

R_2 中已包括第 1 个候选键，因此无需再为第 2 个候选键 (stId, Cno) 单独构造子模式

BCNF VS 3NF

- **Note:** It is always possible to decompose a relation into a set of relations that are in 3NF such that:
 - ▣ decomposition is lossless
 - ▣ dependencies are preserved
- **Note:** It is always possible to decompose a relation into a set of relations that are in BCNF such that:
 - ▣ decomposition is lossless
 - ▣ it **may not** be possible to preserve dependencies.

Appendix 7-1 Computing Candidate Keys

- 定理
 - $R, F=\{\alpha \rightarrow \beta\}$ holds on R
 - 如果属性 A 只出现在 F 中函数依赖的左部 α ，则 A 一定是主属性，必然出现在候选键中
- 输入：关系模式 R 及其函数依赖集 F
- 输出： R 的所有候选关键字
- 方法：
 - step1. 将 R 的所有属性分为四类：
 - L 类：仅出现在 F 中函数依赖左部的属性
 - R 类：仅出现在 F 中函数依赖右部的属性
 - N 类：在 F 中函数依赖左右两边均未出现的属性
 - LR 类：在 F 中函数依赖左右两边均出现的属性

Computing Candidate Keys

- 令 $X_set = L类 \cup N类$, $Y_set = LR类$;
- step2. 求 X_set^+ ,
如果 $X_set^+ = R$, 则 X_set 为 R 的唯一候选键, 转 step5;
否则转 step3
- step3. 在 Y_set 中取一属性 A , 求 $(X_set \cup A)^+$, 若它包含了 R 的所有属性, 则转 step4;
否则, 调换一属性反复进行这一过程, 直到试完所有 Y_set 中的属性
- step4. 如果已找出所有的候选键, 则转 step5;
否则, 在 Y_set 中依此取两个、三个, ..., 求其属性闭包, 直至其闭包包含 R 的所有属性
- step5. 停止, 输出结果

Example 1 求候选键

- $R(X, Y, Z, W)$, and

$$F = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\}$$

- L 类 : Y, W
- R、N: 空
- LR: X, Z
- $X_set = \{Y, W\}$
- $Y_set = \{X, Z\}$
- $(YW)^+ = R$, so YW is the candidate key

- $R(B, D, I, O, Q, S)$, and

$$F = \{B \rightarrow Q, I \rightarrow S, IS \rightarrow Q, S \rightarrow D\}$$

- L 类 : B, I
- R 类 : D, Q
- N 类 : O
- LR 类 : S
- $X_set = \{B, I, O\}$
- $Y_set = \{S\}$
- $(BIO)^+ = R$, so BIO is a candidate key

Example 2 计算候选键

- $R(A, B, C, D, E)$
- $F = \{AB \rightarrow CE, E \rightarrow AB, C \rightarrow D\}$
- Step1. $X_set = \text{空}$, $Y_set = \{A, B, C, E\}$
 - L 类 : 空
 - R 类 : D
 - LR 类 : A, B, C, E
 - N 类 : 空
- Step2. 从 $Y_set = \{A, B, C, E\}$ 中只取 1 个属性 , A, B, C, E ,
 - $(A)^+ \neq R, (B)^+ \neq R, (C)^+ \neq R$, A 、 B 、 C 不是候选键
 - $E \rightarrow R$, E 是一个候选键
 - 且其它任何包含 E 的属性集不可能是候选键
- Step3. 从 Y_set 中取 2 个属性 (不包括 E) , AB, AB $\rightarrow R$; BC, $(BC)^+ \neq R$,
 - AB 是一个候选键 , 且其它任何包含 AB 的属性集不可能是候选键
- Step4. 无法从 Y_set 中取 3 个属性 , 使得这 3 个属性不包括 E 、 不包括 AB
- 因此 , 只有 2 个候选键 E, AB

Example 3 计算候选键

- 给定关系模式 $R(A,B,C,D,E)$ ，以及函数依赖集合 $A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow AB$ ，找出 R 的

全部候选键

- 答案【4 个候选键】

- 出现在左边的属性 $\{A,C,D,B,E\}$ ，出现在右边的属性 $\{A,C,D,B,E\}$
- L 类属性 $\{\}$
- R 属性 $\{\}$
- LR 类双部属性 $\{A,C,D,B,E\}$

- 计算候选键

- 测试 $A^+ = \{A,B,C,D,E\}$, A 是候选键，包含 A 的其他所有属性集合都不是候选键
- 测试 $E^+ = \{A,B,C,D,E\}$, E 是候选键，包含 E 的其他所有属性集合都不是候选键
- 测试 B^+, C^+, D^+ ，都不能包含所有的属性
- 测试 $CD^+ = \{A,B,C,D,E\}$, CD 是候选键，包含 CD 的其他所有属性集合都不是候选键
- 测试 BD^+ 都不能包含所有的属性；
- 测试 $BC^+ = \{A,B,C,D,E\}$, BC 是候选键，包含 BC 的其他所有属性集合都不是候选键

Appendix 7-2 习题类型

- 给定关系表 $r(R)$ 和若干函数依赖 $\alpha \downarrow \beta$ ，判断 r 是否满足 $\alpha \downarrow \beta$
- 根据 Amstron 公理系统，判断关于函数依赖的一些公式是否成立
 - 例如，如果 $A \rightarrow B, B \rightarrow C$ ，则 $A \rightarrow C$
- 根据文字描述，抽象出函数依赖关系
- 设计 SQL 语句，判断函数依赖和键
- 利用算法，求候选键 (Appendix 7-1)
- 计算属性闭包 α^+
- 计算函数依赖集 F 的最小正则集 F_c

Appendix 规范化习题类型

- 给定关系模式 R 和定义在 R 上的函数依赖集 F ，判断 R 属于第几范式，为什么？
 - 只考虑 1NF, 2NF, 3NF, BCNF
- 判断一个模式分解是否为无损连接（定理）、函数依赖保持（定义）
- 给定非 3NF 的关系模式 R 和定义在 R 上的函数依赖集 F ，将 R 分解为第三范式
- 给定非 BCNF 的关系模式 R 和定义在 R 上的函数依赖集 F ，将 R 分解为 BCNF 范式

Functional dependency: deterministic/functional relation among attributes in schema

chapter 7 Relational Database Design

I. Features of good relational design (7.1)

why normalization needed?
good schemas?
(pitfalls in modification on schemas)

decomposition,
lossless decomposition

normalization principles

II. Decomposition using functional dependencies (7.2)

keys and functional dependencies

lossless decomposition and functional
dependencies

III-1. Normal forms: (7.3) standards for good schemas

BCNF

definition

dependency preservation

3NF, 2NF

properties and relationships
among 2NF, 3NF and BCNF

higher normal forms,
not based on FD

III-2. Atomic domains and first normal forms (7.8)

IV. Functional dependency theory (7.4)

closure of FD set

closure of attribute set

canonical cover

dependency preservation

V. Algorithms for decomposition using functional dependencies (7.5)

BCNF decomposition

3NF decomposition

VI[略]. Decomposition using multi-value dependency (7.6) More normal forms (7.7)

VII[略]. Database design process (7.9)

Appendix 7-1 求解候选键算法

Appendix 7-2 Examples and Exercises



Thanks for your
attention